

Documentação do Projeto Gincana (React + Express + Firebase)

Este documento consolida toda a estrutura do projeto, cobrindo desde o setup até o deploy, seguindo boas práticas de desenvolvimento com arquitetura em camadas. O sistema é composto por um frontend em React (Vite) e um backend em Express.js, com banco de dados Firebase (Firestore).

Setup do Projeto

O projeto utiliza **Node.js** e **Express** no backend, **React com Vite** no frontend, e **Firebase (Firestore)** como banco de dados principal.

Dependências do Backend

Pacote	Versão	Função no projeto
bcrypt	5.1.1	Hash de senhas e comparação segura
dotenv	16.6.1	Carrega variáveis de ambiente do arquivo .env
express	4.21.2	Servidor HTTP e roteamento de requisições
firebase-admin	11.11.1	Integração com Firebase / Firestore
jsonwebtoken	9.0.2	Geração e validação de tokens JWT
nodemon	2.0.22	Reinício automático do servidor durante desenvolvimento

Pré-requisitos

- Node.js LTS (≥ 18)
- npm ou yarn
- Git

Instalação

1. Clonar o repositório:

```
git clone <URL>
cd gincana
```

2. Instalar dependências do backend:

```
cd backend  
npm install
```

3. Instalar dependências do frontend:

```
cd ../frontend  
npm install
```

Execução em modo desenvolvimento

Backend (Express.js)

```
cd backend  
npm run dev
```

Frontend (React + Vite)

```
cd frontend  
npm run dev
```



Arquitetura do Projeto

A arquitetura segue o padrão de camadas, separando responsabilidades entre controle, serviço e repositório. O backend foi desenvolvido em **Express.js**, escolhido pela leveza e simplicidade, com Firebase (Firestore) como banco de dados.



Frontend

- React + Vite
- Context API para gerenciamento de estado global
- TailwindCSS para estilização
- React Router para roteamento



Backend

- Express.js (alternativas consideradas: NestJS e Fastify)
- Estrutura modular: controllers, services e repositories
- Autenticação baseada em JWT
- Integração com Firebase (Firestore)



Banco de Dados

- Banco principal: Firebase (Firestore)
- Estrutura de dados ainda em definição



Estrutura do Projeto (Frontend + Backend)

Estrutura

Descrição

backend/

Pasta raiz do backend

├─ src/

Código fonte principal do backend

| └─ config/

Configurações do app, banco, JWT, .env

| └─ routes/

Definição de endpoints

| └─ controllers/

Entrada das requisições

└─ services/	Regras de negócio
└─ repositories/	Acesso ao banco de dados
└─ models/	Entidades/ORM
└─ middlewares/	Auth, validações, tratamento de erros
└─ utils/	Helpers, formatação, funções auxiliares
└─ app.js	Inicialização do servidor
└─ tests/	Testes unitários e de integração
└─ package.json	Configurações do projeto e dependências
└─ .env	Variáveis de ambiente
Estrutura	Descrição
frontend/	Pasta raiz do frontend (React + Vite)
└─ src/	Código fonte principal do frontend
└─ components/	Componentes reutilizáveis
└─ pages/	Páginas e rotas
└─ hooks/	Custom hooks
└─ services/	Chamadas à API / integração com backend
└─ context/	Context API para estado global
└─ assets/	Imagens, ícones, estilos

◆ Boas práticas no frontend

- Componentes pequenos e reutilizáveis
- Pastas por funcionalidade quando necessário
- Padronização com ESLint + Prettier
- Versionamento semântico de commits



Style Guide



Lint e formatação

- ESLint para padronização de código JS/TS
- Prettier para formatação consistente
- EditorConfig para unificar indentação
- Husky + lint-staged para pre-commit



Organização de pastas

gincana/
├── backend/ → API e lógica do servidor
│ └── src/
└── frontend/ → Interface React
 └── src/



Convenções de escrita

- Identação: 2 espaços
- Aspas simples
- Ponto e vírgula omitido
- Imports absolutos
- Nomes de arquivos: kebab-case.js no frontend, PascalCase.ts para componentes React



Nomes de branches

- feature/: novas features
- fix/: correções leves
- hotfix/: correções urgentes



Clonando o Repositório

```
cd existing_repo
git remote add origin https://gitlab.unipampa.edu.br/ales/rp-vi-2025-2/grupo-01.git
git branch -M main
git push -uf origin main
```

Fluxo de Branches

Branch	Finalidade	Origem	Merge para
main	Código de produção	—	—
develop	Integração contínua	main	release
feature/*	Novas funcionalidades	develop	develop
fix/*	Correções não críticas	develop	develop
hotfix/*	Correções urgentes em produção	main	main → develop

 Commits diretos em main e develop são proibidos. Sempre usar Pull Requests.

Commits Semânticos

Formato:
<type>(<scope>): <mensagem curta no imperativo>

Tipos aceitos:

- feat – nova funcionalidade
- fix – correção de bug
- docs – alterações de documentação
- style – ajustes de formatação
- refactor – refatoração
- test – testes
- build – mudanças de dependências

Contribuição

Como contribuir

1. Crie uma branch a partir de develop:
git checkout develop
git pull origin develop
git checkout -b feature/nome-da-feature
2. Desenvolva seguindo o Style Guide.
3. Faça commits semânticos conforme o Workflow.
4. Envie sua branch:
git push origin feature/nome-da-feature
5. Abra um Pull Request.

Checklist do PR

- Código formatado (ESLint/Prettier)
- Testes rodando localmente
- Documentação atualizada
- Nenhum erro crítico no lint



Deploy



Processo

1. Build do frontend (npm run build)
2. Build do backend (npm run build)
3. Configuração de variáveis de ambiente (.env)
4. Deploy em ambiente de homologação
5. Aprovação → Deploy em produção



Ambientes

- Dev: rodando localmente ou em containers
- Homolog: testes internos
- Prod: ambiente final para usuários



Monitoramento e Logs

- Log de erros (Sentry, LogRocket)
- Monitoramento (Grafana, Prometheus)
- Alertas de f